

Reviewer Pack — Minimal Order of Quasi-Crystalline Sheaves and Resolution Pr...

Entry eb6e20bf88b0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 22:56:38 UTC

Minimal Order of Quasi-Crystalline Sheaves and Resolution Proof Width Inequality

Entry ID: eb6e20bf88b0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 22:56:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quasi-Crystalline Sheaves) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the resolution proof width $w(\varphi)$ satisfies $|w(\varphi)| \leq O(\log(n) * \min_order(QC(\varphi)))$, where $QC(\varphi)$ denotes the minimum order of a quasi-crystalline sheaf associated with φ .

Rationale (proposer's reasoning):

Quasi-crystalline sheaves provide a geometric interpretation of complexity measures, potentially revealing hidden structures in proof complexity. If the conjecture holds, it suggests a deep connection between arithmetic geometry and resolution proofs that could lead to new insights into computational hardness.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 162cc82b80605ce7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all CNFs φ with n variables ($n \leq 40$), the resolution proof width $w(\varphi)$ satisfies $|w(\varphi)| \leq O(\log(n) * \min_order(QC(\varphi)))$ and at least 80% of the seeds produce this condition, where $QC(\varphi)$ is the minimum order of a quasi-crystalline sheaf associated with φ .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND quasi-crystalline sheaves AND resolution proof complexity - resolution proof width inequality AND min_order quasi-crystalline sheaf - CNF $\emptyset$ AND minimum order quasi-crystalline sheaf

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.7s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-1, 0) * (i + 1) for i in range(n)]
            if all(c == 0 for c in clause):
                clause[random.randint(0, n-1)] = random.choice([-1, 1])
            clauses.append(clause)
        return clauses

    def quasi_crystalline_sheaf(cnf):
        n = len(cnf[0]) // 2
        matrix = [[0] * (2**n) for _ in range(2**n)]
        for clause in cnf:
            for i in range(n):
                if clause[i] != 0:
                    row = int(''.join(str(abs(x)) for x in clause[:i]), 2)
                    col = int(''.join(str(abs(x)) for x in clause[i+1:]), 2)
                    matrix[row][col] += abs(clause[i])
        return matrix

    def min_order(matrix):
        n = len(matrix)
        visited = [False] * n
        order = 0

        def dfs(node):
```

```

        nonlocal order
        stack = [node]
        while stack:
            node = stack.pop()
            if not visited[node]:
                visited[node] = True
                order += 1
                for neighbor in range(n):
                    if matrix[node][neighbor] != 0 and not visited[neighbor]:
                        stack.append(neighbor)

        for i in range(n):
            if not visited[i]:
                dfs(i)
        return order

def resolution_width(cnf):
    n = len(cnf[0]) // 2
    queue = cnf[:]
    width = 1

    while queue:
        new_queue = []
        for clause in queue:
            for i in range(n):
                if clause[i] != 0:
                    new_clause = [x for x in clause[:i]] + [-x for x in clause[i+1:]]
                    if any(new_clause == c for c in cnf):
                        continue
                    new_queue.append(new_clause)
        queue = new_queue
        width += len(queue)
    return width

n = random.randint(5, 40)
cnf = generate_cnf(n)
qc_sheaf = quasi_crystalline_sheaf(cnf)
min_order_qc = min_order(qc_sheaf)
w_phi = resolution_width(cnf)

return {
    "metric_name": "resolution_width",
    "metric_value": abs(w_phi),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(w_phi) <= Fraction(n).log(2) * min_order_qc,
    "counterexample": "" if conjecture_holds else f"n={n}, w_phi={w_phi}, min_order_qc={min_order_qc}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)

```

```

print(f"TRIAL: {result}")
results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\n={results[0]['n_max']}, w_phi={results[0]['metric_value']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13890
2	propose	ollama_remote	glm4:latest	0	0	13801
3	preregistration	ollama_remote	glm4:latest	0	0	9541
4	novelty	ollama_remote	glm4:latest	0	0	15356
5	novelty	ollama_remote	glm4:latest	0	0	8781
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17768
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12221
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12409
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11653
10	judge	ollama_remote	glm4:latest	0	0	68797

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184218 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/eb6e20bf88b0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/eb6e20bf88b0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/eb6e20bf88b0.tar.gz (if generated)